ИКОНОМИЧЕСКИ УНИВЕРСИТЕТ - ВАРНА

ФАКУЛТЕТ “ИНФОРМАТИКА”



КУРСОВА РАБОТА

по дисциплината

Програмиране с Python

тема: Магазин за дрехи

Разработил:

Синем Неждет, 126000, 34 гр.

Проверили:

гл. ас. д-р Стойчо Стоев

х. ас. Георги Пенчев

ВАРНА

2026

Съдържание

1. Описание на фирмата 3
2. Описание на задачите 3
2.1. Прочитане на данни от файлове 3
2.2. Прехвърляне на данните от файловете в списъци и визуализирането им в интерфейса 3
2.3. Попълване на речник от списъците с обобщаване на данните и визуализирането им . 5
2.4. Търсене на данни от речника и извеждане на резултатите 6
2.5. Прехвърляне на данни по определен критерий в нов списък и извеждането му 6
2.6. Запис на данните от списъка във файл в подходящ формат 7
2.7. Добавяне на нови данни чрез интерфейса към структура от данни на python 8
2.8. Интерфейс с бутони, listbox и др. уиджети 12
2.9. Визуализация на данните с графика с matplotlib 13

1. Описание на фирмата

„ЕГ-ШОП“ ЕООД е съвременна търговска фирма, опериращa в сектора на бързооборотните модни стоки чрез своя специализиран онлайн магазин за дрехи „ЕЛЕА“. Основната дейност на фирмата включва внос, дистрибуция и търговия на дребно с облекло (тениски, дънки, якета и др.). Поради нарастващия обем от продажби, фирмата внедрява настоящата софтуерна система за управление. Приложението позволява на мениджмънта да следи в реално време наличностите в склада, да филтрира стоките по ценови критерии за маркетингови кампании, безопасно да регистрира нови артикули и продажби, както и да извършва автоматични финансови справки и графични анализи на оборота по категории стоки за целите на счетоводството.

2. Описание на задачите

2.1. Прочитане на данни от файлове

Програмата прочита три външни Excel файла, трансформира суровите редове в структурирани обекти (речници, множество и списъци).

```
    products = list()
    sales = list()
    categories = dict()
    valid_codes = set()
```

```
def load_data():
    wb_cat = op.load_workbook("categories.xlsx")
    ws_cat = wb_cat.active

    for cat_id, cat_name in ws_cat.iter_rows(min_row = 2, values_only=True):
        if cat_id:
            categories[str(cat_id)] = str(cat_name)

    wb_prod = op.load_workbook("products.xlsx")
    ws_prod = wb_prod.active

    for prod_id, cat_id, model, size, price in ws_prod.iter_rows(min_row = 2, values_only=True):
        if prod_id:
            products.append({
                'id':str(prod_id),
                'cat_id':str(cat_id),
                'model':str(model),
                'size':str(size),
                'price':float(price)
            })
            valid_codes.add(str(prod_id))

    wb_sales = op.load_workbook("sales.xlsx")
    ws_sales = wb_sales.active

    for prod_id, date, qty in ws_sales.iter_rows(min_row=2, values_only=True):
        if prod_id:
            sales.append({
                'id':str(prod_id),
                'date':str(date),
                'qty':int(qty)
            })
```

2.2. Прехвърляне на данните от файловете в списъци и визуализирането им в интерфейса

Програмата извлича информация от динамичния списък products и я визуализира в графичен прозорец чрез табличния компонент Treeview.

```
def view_menu():
    view_root = tk.Tk()
    view_root.title("Наличности в магазина")
    view_root.geometry("600x500")

    columns = ('Код', 'Категория', 'Модел', 'Размер', 'Цена')
    tree = ttk.Treeview(view_root, columns=columns, show="headings", height=12)
    tree.heading('Код', text = 'Код')
    tree.heading('Категория', text = 'Категория')
    tree.heading('Модел', text = 'Модел')
    tree.heading('Размер', text = 'Размер')
    tree.heading('Цена', text = 'Цена')

    tree.column('Код', width=80, anchor="center")
    tree.column('Категория', width=120, anchor="center")
    tree.column('Модел', width=180, anchor="center")
    tree.column('Размер', width=80, anchor="center")
    tree.column('Цена', width=100, anchor="center")

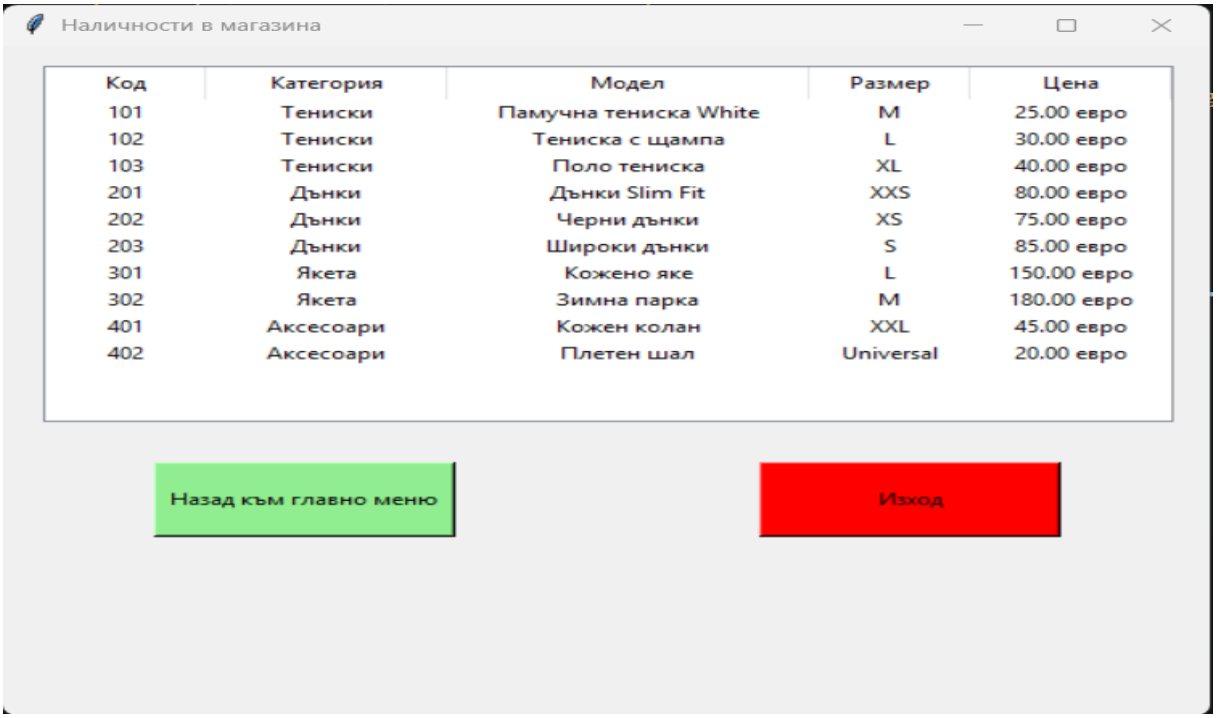
    tree.grid(row=0, column=0, columnspan=2, padx=20, pady=15)
    btn_back = tk.Button(view_root, text="Назад към главно меню", width=20, height=3, bg = "lightgreen")
    btn_exit = tk.Button(view_root, text="Изход", width=20, height=3, bg = "red")
    btn_back.grid(row=1, column=0, padx=20, pady=15)
    btn_exit.grid(row=1, column=1, padx=20, pady=15)

    def fill_table():
        for p in products:
            cat_name = categories.get(p['cat_id'], "Неизвестна")
            tree.insert("", tk.END, values=(p['id'], cat_name, p['model'], p['size'], f"{p['price']:.2f} евро"))
```

```
def go_back():
    view_root.destroy()
    main_menu()

fill_table()
btn_back.config(command=go_back)
btn_exit.config(command=view_root.quit)

view_root.mainloop()
```



2.3. Попълване на речник от списъците с обобщаване на данните и визуализирането им

Този модул обхожда направените продажби и цените на продуктите, като автоматично пресмята и обобщава финансовия оборот по групи стоки в нов речник, след което показва крайния финансов резултат на екрана.

```
def reports_menu():
    reports_root = tk.Tk()
    reports_root.title("Справки и графики")
    reports_root.geometry("400x400")

    tk.Label(reports_root, text="Финансови справки за магазина", font="bold").grid(row=0, column=0, columnspan=2, pady=15)

    btn_calc = tk.Button(reports_root, text="Изчисли оборота", width=22, bg = "lightblue")
    btn_calc.grid(row=1, column=0, padx=15, pady=10)

    btn_chart = tk.Button(reports_root, text="Покажи графика", width=22, bg="lightblue")
    btn_chart.grid(row=1, column=1, padx=15, pady=10)

    listbox_rep = tk.Listbox(reports_root, width=55, height=12)
    listbox_rep.grid(row=2, column=0, columnspan=2, padx=15, pady=10)

    btn_back = tk.Button(reports_root, text="Назад към менюто", bg="lightgreen", width=15)
    btn_back.grid(row=3, column=0, padx=15, pady=20, sticky="w")

    btn_exit = tk.Button(reports_root, text="Изход", bg="red", width=15)
    btn_exit.grid(row=3, column=1, padx=15, pady=20, sticky="e")

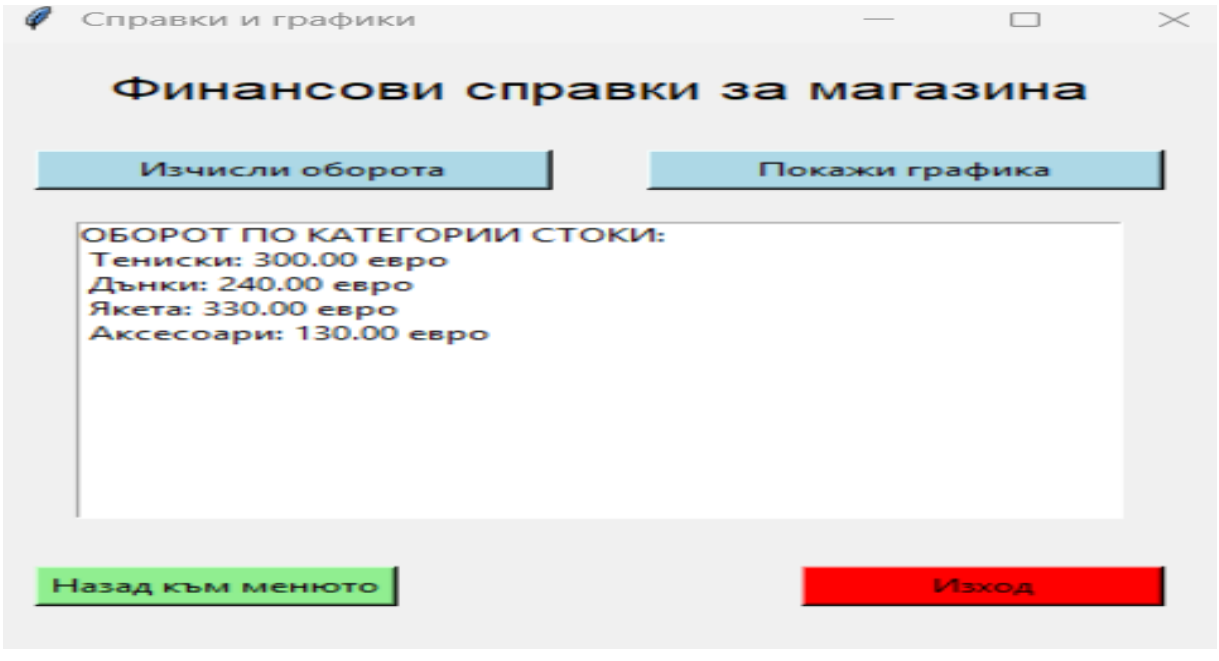
    category_sum = dict()
```

```
def reports_menu():
    def run_calculations():
        category_sum.clear()
        for cat_name in categories.values():
            category_sum[cat_name] = 0.0

        for s in sales:
            prod_id = s['id']
            qty = s['qty']
            for p in products:
                if p['id'] == prod_id:
                    price = p['price']
                    cat_name = categories.get(p['cat_id'], "Неизвестна")
                    amount = qty * price
                    category_sum[cat_name] += amount

    def calculate_reports():
        listbox_rep.delete(0, tk.END)
        run_calculations()

        listbox_rep.insert(tk.END, "ОБОРОТ ПО КАТЕГОРИИ СТОКИ:")
        for cat_name, total_sum in category_sum.items():
            listbox_rep.insert(tk.END, f" {cat_name}: {total_sum:.2f} евро")
```



2.4. Търсене на данни от речника и извеждане на резултатите

Функцията позволява на администратора бързо да провери наименованието на дадена категория стоки директно от речника чрез въвеждане на нейния уникален код.

```
def search_menu():
    search_root = tk.Tk()
    search_root.title("Търсене и Филтриране")
    search_root.geometry("470x450")

    lbl_search_cat = tk.Label(search_root, text="Въведете код на категория (1-4):")
    e_cat_id = tk.Entry(search_root, width=10)
    btn_search_cat = tk.Button(search_root, text="Търси категория", width = 17, bg="lightblue")
    lbl_cat_result = tk.Label(search_root, text="Резултат: ")

    lbl_search_cat.grid(row=0, column=0, sticky="w", padx=15, pady=10)
    e_cat_id.grid(row=0, column=1, sticky="w", padx=5, pady=10)
    btn_search_cat.grid(row=0, column=2, sticky="w", padx=5, pady=10)
    lbl_cat_result.grid(row=1, column=0, columnspan=3, sticky="w", padx=15, pady=5)

    lbl_filter_price = tk.Label(search_root, text="Покажи стоки с цена над (евро):")
    e_price_limit = tk.Entry(search_root, width=10)
    btn_filter = tk.Button(search_root, text="Филтрирай", width = 17, bg="lightblue")
    lbl_listbox = tk.Label(search_root, text="Резултати от филтрирането:")
    listbox = tk.Listbox(search_root, width=60, height=10)

    lbl_filter_price.grid(row=2, column=0, sticky="w", padx=15, pady=10)
    e_price_limit.grid(row=2, column=1, sticky="w", padx=5, pady=10)
    btn_filter.grid(row=2, column=2, sticky="w", padx=5, pady=10)
    lbl_listbox.grid(row=3, column=0, sticky="w", padx=15, pady=5)
    listbox.grid(row=4, column=0, columnspan=3, padx=15, pady=5)

    btn_save_file = tk.Button(search_root, text="Запиши в текстов файл", width = 20, bg="lightblue")
    btn_back = tk.Button(search_root, text="Назад към менюто", width=20, bg="lightgreen")
    btn_exit = tk.Button(search_root, text="Изход", width=20, bg="red")
```

```
    btn_save_file.grid(row=5, column=0, columnspan=3, pady=10)
    btn_back.grid(row=6, column=0, padx=15, pady=20, sticky="w")
    btn_exit.grid(row=6, column=2, padx=15, pady=20, sticky="e")

    filtered_products = list()

    def search_category():
        search_key = e_cat_id.get().strip()
        if search_key in categories:
            lbl_cat_result.config(text=f"Резултат: Код {search_key} - {categories[search_key]}", fg="green")
        else:
            lbl_cat_result.config(text="Резултат: Няма такава категория!", fg="red")
```



2.5. Прехвърляне на данни по определен критерий в нов списък и извеждането му

Програмата позволява въвеждане на цена и филтрира само онези продукти, чиято цена надвишава зададения от потребителя лимит, като ги добавя в нов списък filtered_products.

```python
def filter_price():
    filtered_products.clear()
    listbox.delete(0, tk.END)

    user_input = e_price_limit.get().strip()
    if not user_input:
        messagebox.showwarning("Внимание", "Моля, въведете цена!")
        return

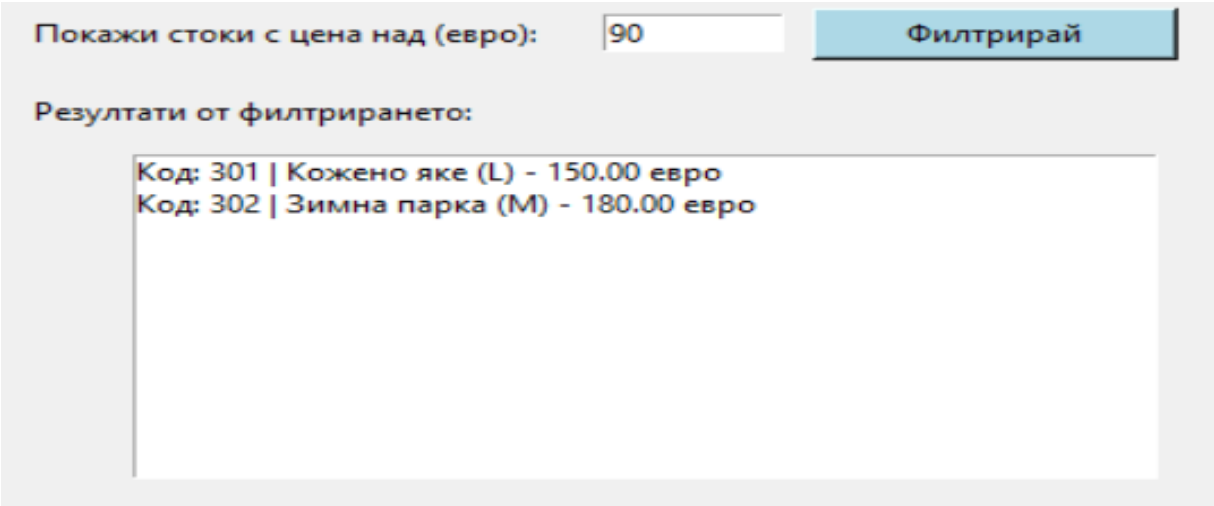
    if not user_input.replace('.', '', 1).isdigit():
        messagebox.showerror("Грешка", "Моля, въведете валидно число за цена!")
        return

    limit = float(user_input)

    for p in products:
        if p['price'] > limit:
            filtered_products.append(p)
```

```python
def filter_price():
            filtered_products.append(p)
            info_text = f"Код: {p['id']} | {p['model']} ({p['size']}) - {p['price']:.2f} евро"
            listbox.insert(tk.END, info_text)

    if len(filtered_products) == 0:
        listbox.insert(tk.END, "Няма намерени стоки с по-висока цена.")
```



2.6. Запис на данните от списъка във файл в подходящ формат

Модулът експортира данни от филтъра от и ги записва във външен текстов файл (filtered_products.txt).

```python
def save_to_file():
    if len(filtered_products) == 0:
        messagebox.showwarning("Внимание", "Списъкът е празен!")
        return

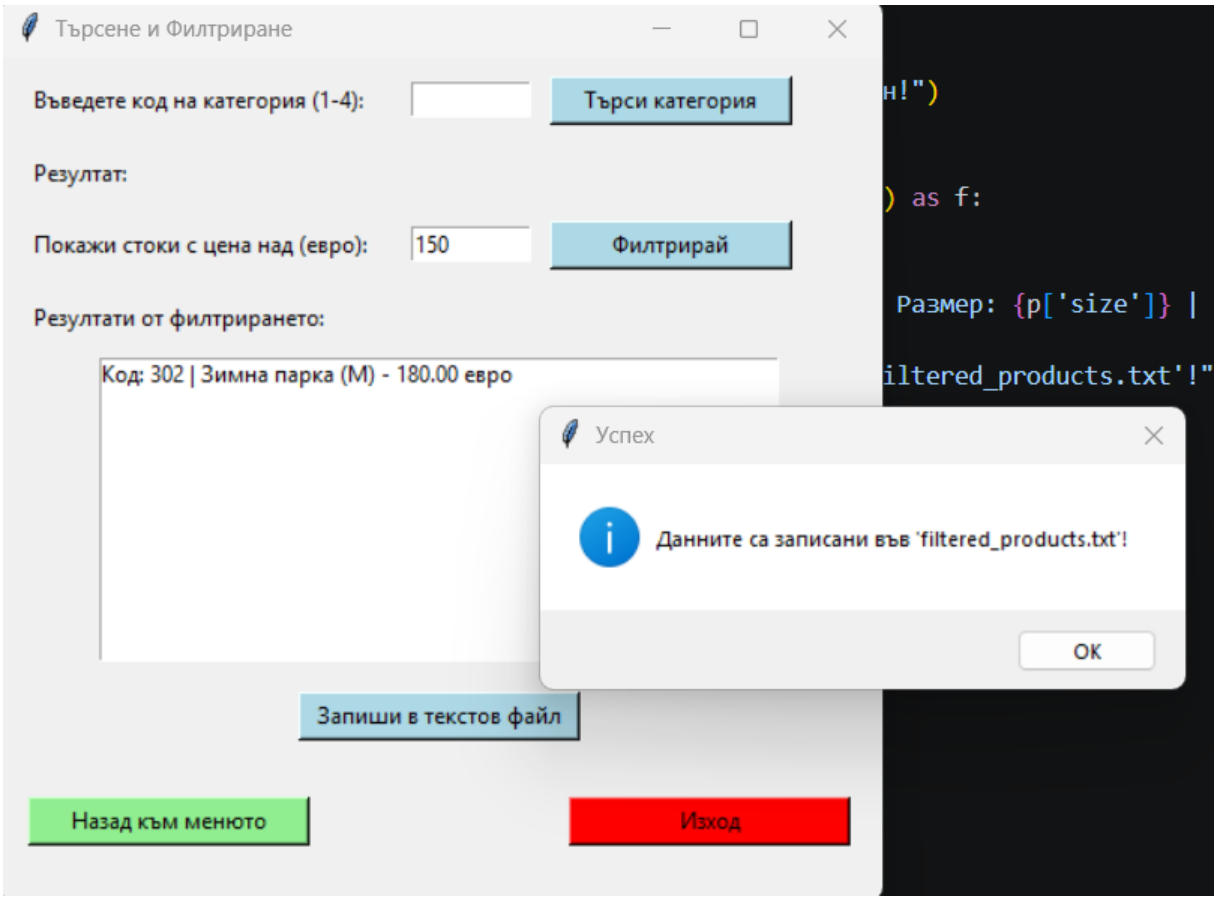
    with open("filtered_products.txt", "w", encoding="utf-8") as f:
        f.write("СПИСЪК С ФИЛТРИРАНИ СТОКИ\n")
        for p in filtered_products:
            f.write(f"Код: {p['id']} | Модел: {p['model']} | Размер: {p['size']} | Цена: {p['price']:.2f} евро\n")

    messagebox.showinfo("Успех", "Данните са записани във 'filtered_products.txt'!")

def go_back():
    search_root.destroy()
    main_menu()

btn_search_cat.config(command=search_category)
btn_filter.config(command=filter_price)
btn_save_file.config(command=save_to_file)
btn_back.config(command=go_back)
btn_exit.config(command=search_root.quit)

search_root.mainloop()
```




```
СПИСЪК С ФИЛТРИРАНИ СТОКИ
Код: 302 | Модел: Зимна парка | Размер: M | Цена: 180.00 евро
```

2.7. Добавяне на нови данни чрез интерфейса към структура от данни на python

Този модул позволява въвеждане на нови стоки и продажби, довавя към списъци и визуализира, като притежава валидация, която проверява дали полетата не са празни и дали са въведени правилни числа.

```
def add_menu():
    add_root = tk.Tk()
    add_root.title("Добавяне на данни")
    add_root.geometry("400x500")

    tk.Label(add_root, text="ДОБАВЯНЕ НА НОВА СТОКА", font="bold").grid(row=0, column=0, columnspan=2, pady=10)

    tk.Label(add_root, text="Код стока:").grid(row=1, column=0, sticky="w", padx=10, pady=5)
    e_new_id = tk.Entry(add_root, width=15)
    e_new_id.grid(row=1, column=1, sticky="w", padx=10)

    tk.Label(add_root, text="Код категория (1-4):").grid(row=2, column=0, sticky="w", padx=10, pady=5)
    e_new_cat = tk.Entry(add_root, width=15)
    e_new_cat.grid(row=2, column=1, sticky="w", padx=10)

    tk.Label(add_root, text="Модел/Име:").grid(row=3, column=0, sticky="w", padx=10, pady=5)
    e_new_model = tk.Entry(add_root, width=15)
    e_new_model.grid(row=3, column=1, sticky="w", padx=10)

    tk.Label(add_root, text="Размер (S, M, L, XL):").grid(row=4, column=0, sticky="w", padx=10, pady=5)
    e_new_size = tk.Entry(add_root, width=15)
    e_new_size.grid(row=4, column=1, sticky="w", padx=10)

    tk.Label(add_root, text="Цена (евро):").grid(row=5, column=0, sticky="w", padx=10, pady=5)
    e_new_price = tk.Entry(add_root, width=15)
    e_new_price.grid(row=5, column=1, sticky="w", padx=10)

    btn_add_prod = tk.Button(add_root, text="Добави стока", bg="lightblue", width=20)
    btn_add_prod.grid(row=6, column=0, columnspan=2, pady=10)

    tk.Label(add_root, text="РЕГИСТРИРАНЕ НА ПРОДАЖБА", font="bold").grid(row=7, column=0, columnspan=2, pady=15)

    tk.Label(add_root, text="Код на продадена стока:").grid(row=8, column=0, sticky="w", padx=10, pady=5)
```

```
def add_menu():
    def add_product():
        nid = e_new_id.get().strip()
        ncat = e_new_cat.get().strip()
        nmodel = e_new_model.get().strip()
        nsize = e_new_size.get().strip()
        nprice = e_new_price.get().strip()

        if not (nid and ncat and nmodel and nsize and nprice):
            messagebox.showwarning("Грешка", "Моля, попълнете всички полета за стоката!")
            return

        if not nid.isdigit():
            messagebox.showerror("Грешка", "Кодът на стоката трябва да бъде цяло число!")
            return

        if not ncat.isdigit():
            messagebox.showerror("Грешка", "Кодът на категорията трябва да бъде цяло число!")
            return

        if nid in valid_codes:
            messagebox.showerror("Грешка", f"Продукт с код {nid} вече съществува!")
            return

        if not nprice.replace('.', '', 1).isdigit():
            messagebox.showerror("Грешка", "Цената трябва да бъде валидно число (напр. 25.50)!")
            return

        if ncat not in categories:
            answer = messagebox.askquestion("Нова категория", f"Категория с код {ncat} не съществува. Искате ли да я създадете?")
            if answer == 'yes':
                categories[ncat] = f"Група {ncat}"
                messagebox.showinfo("Успех", f"Категория {ncat} беше добавена успешно!")
```

```
    def add_product():
            else:
                return

        products.append({
            'id': nid, 'cat_id': ncat, 'model': nmodel, 'size': nsize, 'price': float(nprice)
        })
        valid_codes.add(nid)
        messagebox.showinfo("Успех", f"Продуктът '{nmodel}' е добавен успешно!")

        e_new_id.delete(0, tk.END); e_new_cat.delete(0, tk.END)
        e_new_model.delete(0, tk.END); e_new_size.delete(0, tk.END); e_new_price.delete(0, tk.END)

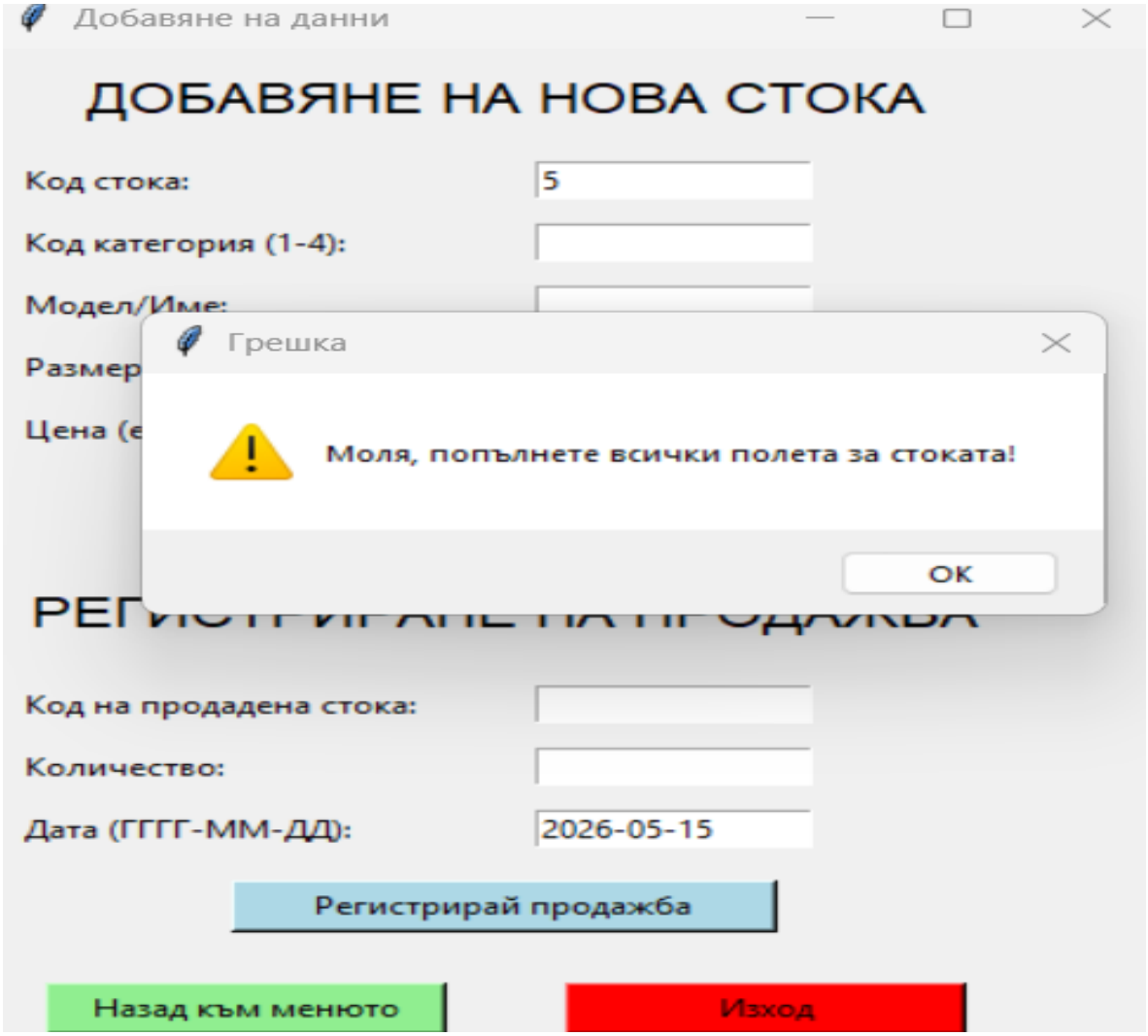
    def add_sale():
        sid = e_sale_id.get().strip()
        sqty = e_sale_qty.get().strip()
        sdate = e_sale_date.get().strip()

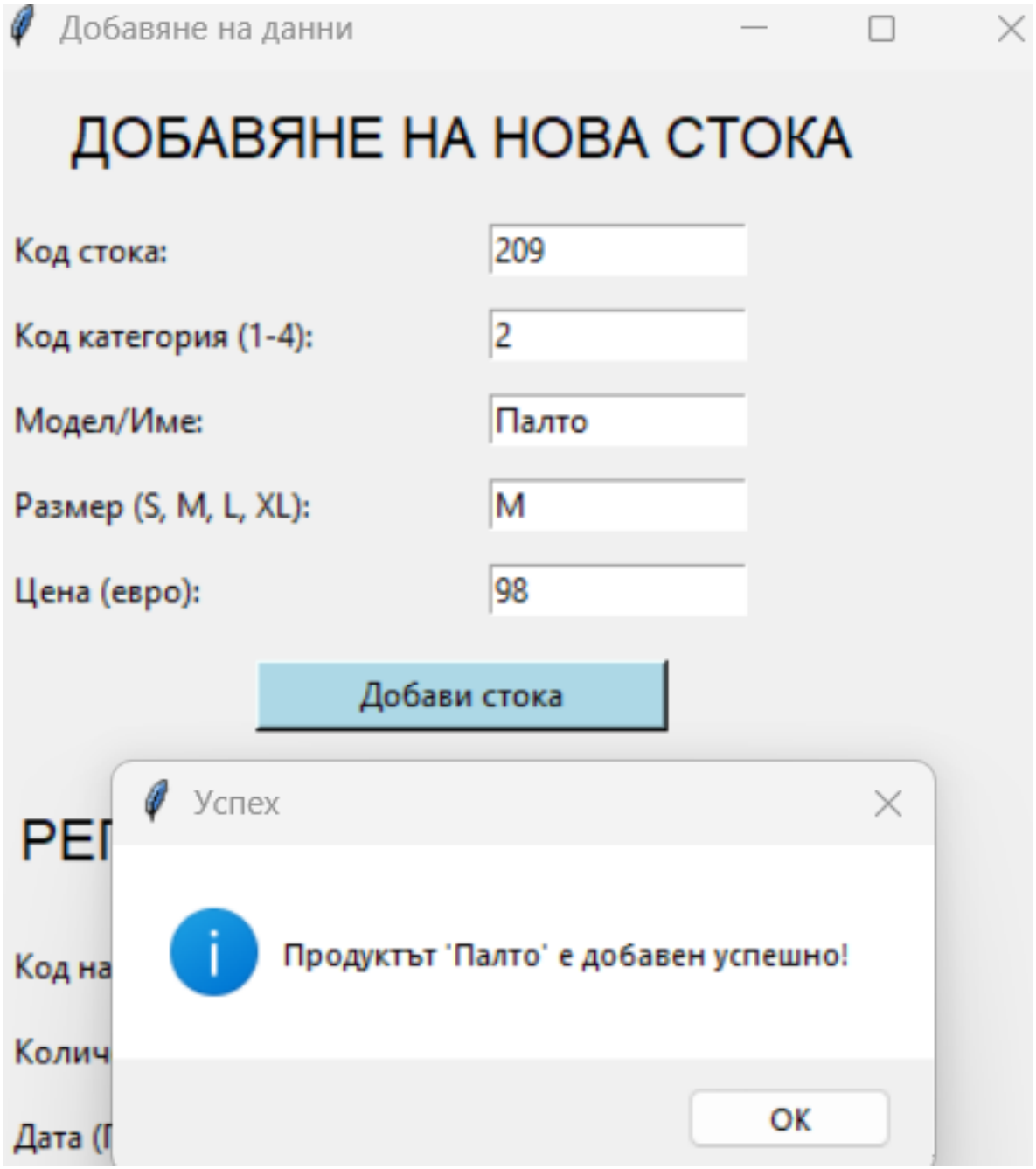
        if not (sid and sqty and sdate):
            messagebox.showwarning("Грешка", "Попълнете всички полета за продажбата!")
            return

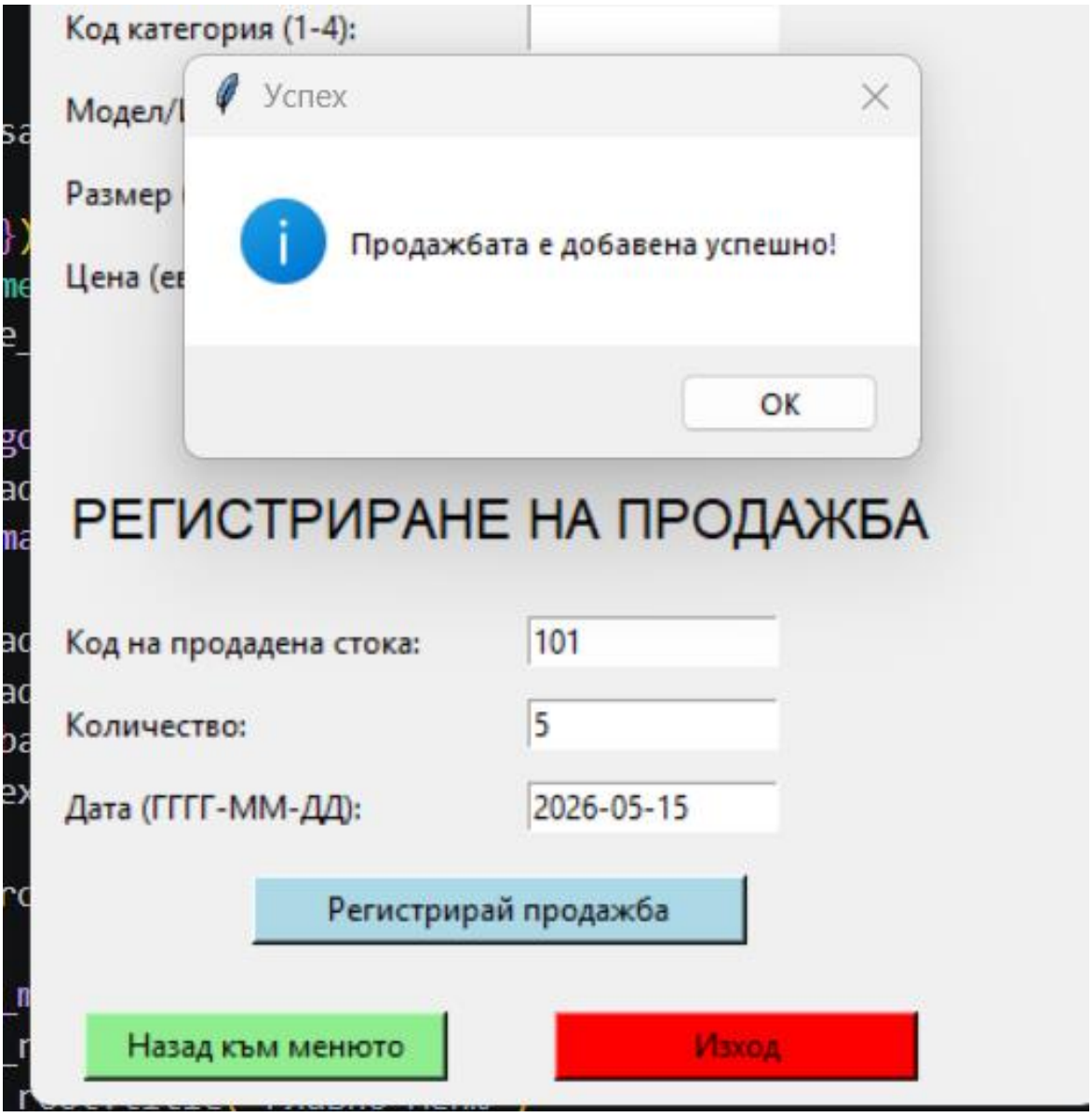
        if sid not in valid_codes:
            messagebox.showerror("Грешка", f"Продукт с код {sid} не съществува! Не може да се направи продажба.")
            return

        if not sqty.isdigit():
            messagebox.showerror("Грешка", "Количеството трябва да бъде цяло число!")
            return

        sales.append({
            'id': sid, 'date': sdate, 'qty': int(sqty)
```


Добавяне на данни
ДОБАВЯНЕ НА НОВА СТОКА
Код стока:
5
Код категория (1-4):
Модел/Име:
Размер
Цена (е
Грешка
Моля, попълнете всички полета за стоката!
OK
РЕГИСТРИРАНЕ НА ПРОДАЖБА
Код на продадена стока:
Количество:
Дата (ГГГГ-ММ-ДД):
2026-05-15
Регистрирай продажба
Назад към менюто
Изход


Добавяне на данни
ДОБАВЯНЕ НА НОВА СТОКА
Код стока:
209
Код категория (1-4):
2
Модел/Име:
Палто
Размер (S, M, L, XL):
M
Цена (евро):
98
Добави стока
Успех
Продуктът 'Палто' е добавен успешно!
OK

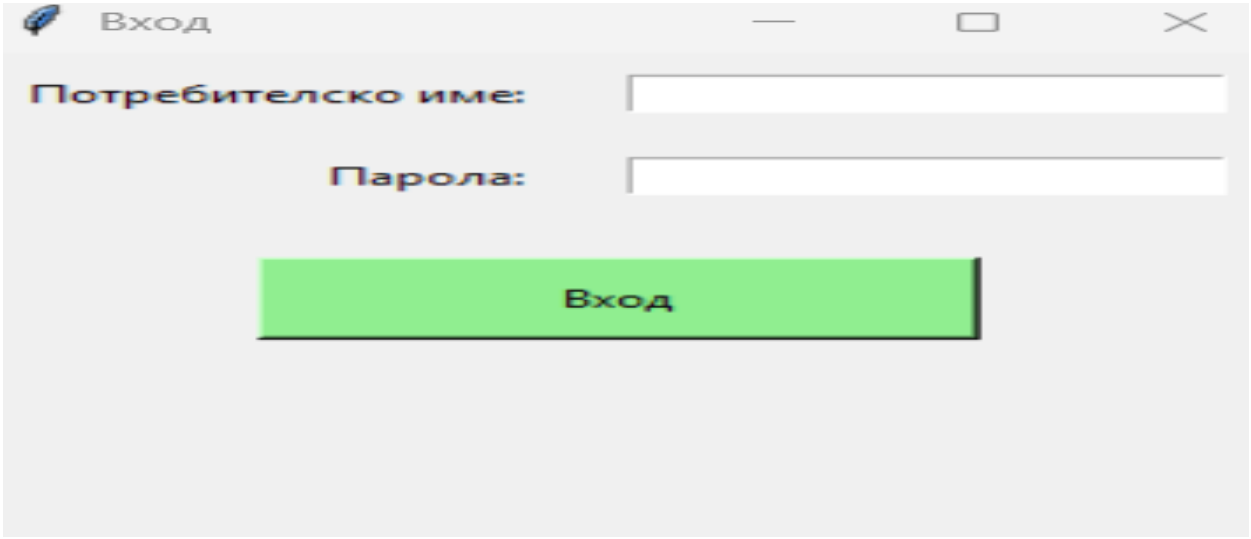


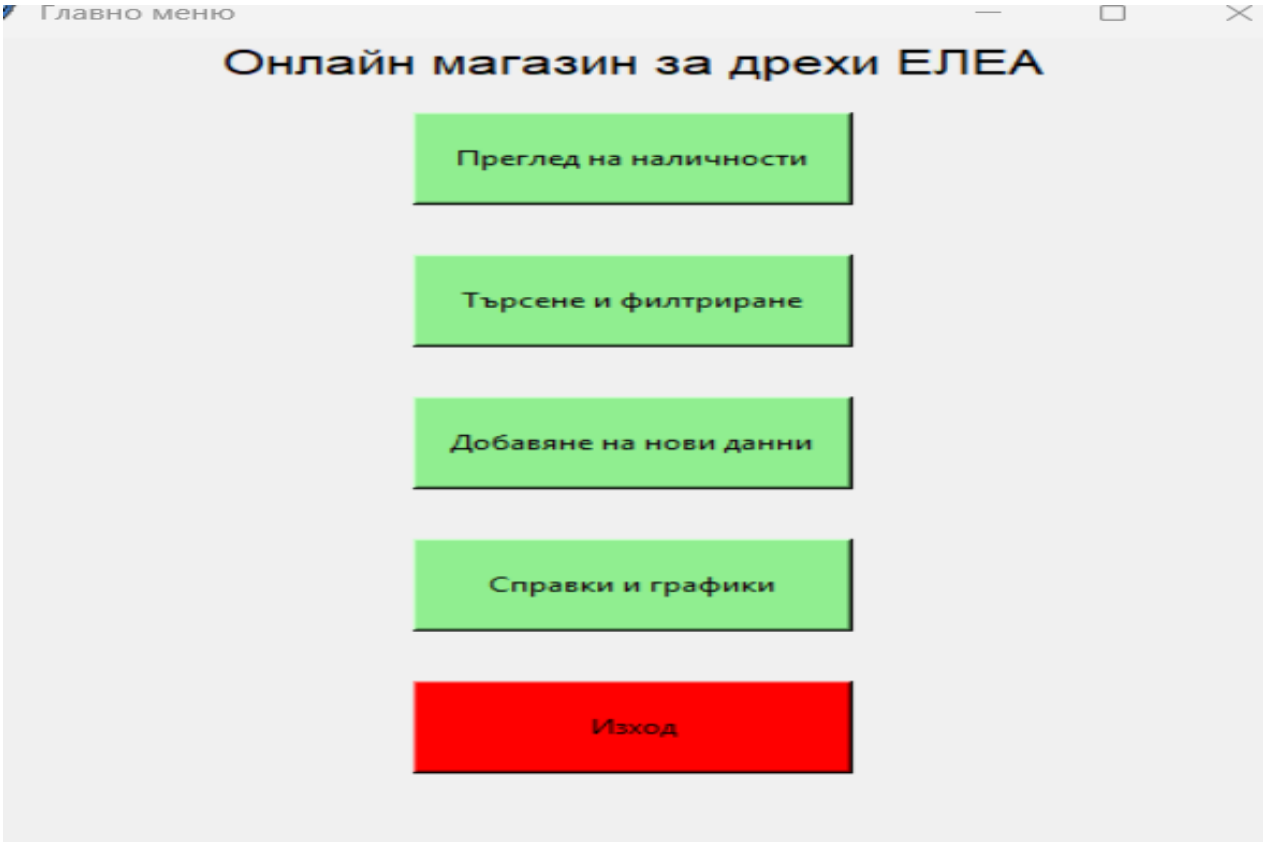
2.8. Интерфейс с бутони, listbox и др. уиджети

Ераните съдържат бутони, listbox, текстови, полета, таблици.

```
tk.Label(main_root, text="Онлайн магазин за дрехи ЕЛЕА", font="bold").pack()
view_btn = tk.Button(main_root, text="Преглед на наличности", width=22, height=3, bg="lightgreen")
search_btn = tk.Button(main_root, text="Търсене и филтриране", width=22, height=3, bg="lightgreen")
add_btn = tk.Button(main_root, text="Добавяне на нови данни", width=22, height=3, bg="lightgreen")
reports_btn = tk.Button(main_root, text="Справки и графики", width=22, height=3, bg="lightgreen")
exit_btn = tk.Button(main_root, text="Изход", command=main_root.quit, width=22, height=3, bg="red")

view_btn.pack(pady=15)
search_btn.pack(pady=15)
add_btn.pack(pady=15)
reports_btn.pack(pady=15)
exit_btn.pack(pady=15)
```





2.9. Визуализация на данните с графика с matplotlib

Модулът превръща обобщените финансови данни от речника в ясна стълбовидна графика чрез Matplotlib, като поставя числови етикети с точност над всеки стълб.

```python
def show_chart():
    run_calculations()

    if sum(category_sum.values()) == 0:
        messagebox.showwarning("Внимание", "Няма регистрирани продажби за показване на графика!")
        return

    bars = plt.bar(list(category_sum.keys()), list(category_sum.values()), color="lightblue")
    plt.bar_label(bars, labels=[f"{val:.2f} евро" for val in category_sum.values()])

    plt.title("Общ оборот по категории стоки (в евро)")
    plt.xlabel("Категории")
    plt.ylabel("Оборот (евро)")
    plt.tight_layout()
    plt.show()

def go_back():
    reports_root.destroy()
    main_menu()
```

